

LC 3414 - Maximum Score of Non-overlapping Intervals

DP notes - Memoized (top-down) & Bottom-Up (tabulation)

1. Problem

LeetCode 3414 - Maximum Score of Non-overlapping Intervals. Given a list of intervals, each with a [start, end, weight], choose **exactly 4** intervals such that no two chosen intervals overlap, and the sum of their weights is maximized. If several sets of 4 give the same maximum sum, return the one whose sorted list of original indices is lexicographically smallest.

2. Intuition

What are we trying to calculate? The best possible total weight from picking exactly 4 non-overlapping intervals, plus which exact intervals (by original index) achieve it.

Obvious / brute-force thought: Try every combination of 4 intervals out of n , check that all pairs are non-overlapping, keep the best sum. That's roughly $O(n^4)$ combinations - way too slow once n grows.

Observation that unlocks it: Sort all intervals by start time. Once an interval is picked, the only intervals that can legally come after it are the ones whose start is strictly greater than its end. Because the array is sorted by start, that "first legal next interval" is just a binary search (**nextIdx[i]**). So instead of checking every pair for overlap, picking interval i simply means "jump to nextIdx[i] for the next decision." The overlap constraint disappears into a single jump.

Why DP fits: After sorting, the only two things that decide the rest of the problem at any point are: *where am I in the sorted list* (i) and *how many more intervals do I still need to pick* (k). Everything before position i is already decided and irrelevant. Many different decision paths arrive at the same (i, k) - that's overlapping subproblems - and the best answer from (i, k) doesn't depend on how we got there - that's optimal substructure. Classic "pick or skip, sorted + jump-ahead" DP.

3. DP State

dp[i][k] (or **solve(i, k)** in the memo version) = the best result obtainable using only intervals from sorted position i onward, while still needing to pick exactly k more intervals. The result stored is a small struct **Node { score, idxs }** - the max achievable weight sum, and the sorted list of original indices that achieve it.

Why (i, k) is enough: i tells us which intervals are still "available" (everything before i is either already used or permanently skipped), and k tells us how many more picks are owed. Nothing else about the path taken to reach (i, k) affects what the best continuation looks like - so (i, k) fully captures the sub-problem. i ranges over $0..n$ and k ranges over $0..4$ (4 is fixed by the problem).

4. Transition

At state (i, k) , for the current interval i (weight wt , original index idx , jump target $j = \text{nextIdx}[i]$):

- **skip** = $\text{dp}[i+1][k]$ — don't use interval i , move to the next one in sorted order.
- **take** = $\text{dp}[j][k-1]$ with wt added to the score and idx inserted into the index list — use interval i , and the next decision can only start from j since anything between i and j would overlap it.
- $\text{dp}[i][k]$ = whichever of skip/take has the higher score. If scores tie, the one with the lexicographically smaller sorted index list wins (this directly satisfies the problem's tie-breaking rule).

Base case: $k == 0$ (no more picks needed) or $i == n$ (ran out of intervals) both return an empty result. This is correct because either there's nothing left to add ($k == 0$) or there is nothing left to add it from ($i == n$).

This is correct because sorting removed the need to ever check overlap between non-adjacent picks - by construction, taking i and then only ever considering positions $\geq \text{nextIdx}[i]$ guarantees every pair among the final chosen intervals is non-overlapping.

5. Memoized / Top-Down (memo.cpp)

Recursive flow: $\text{solve}(i, k)$ directly mirrors the transition above - it computes *skip* by recursing to $\text{solve}(i+1, k)$, computes *take* by recursing to $\text{solve}(\text{nextIdx}[i], k-1)$ and adding the current interval's weight/index, then returns the better of the two (with the tie-break rule).

Base case: "if ($k == 0 \parallel i \geq n$) return $\text{Node}()$;" - a fresh $\text{Node}()$ has score = -1 and empty idxs, representing "nothing more to add."

Memoization: "if ($\text{dp}[i][k].\text{score} \neq -1$) return $\text{dp}[i][k]$;" reuses a previously computed answer for that exact (i, k) pair instead of recomputing it, which is what turns the exponential recursion tree into $O(n * k)$ actual work.

How the recurrence is implemented: after computing skip and take Nodes, take's idxs vector is re-sorted (since the current index is pushed after copying temp.idxs) so that comparisons and the final answer stay in sorted order; the winner is cached into $\text{dp}[i][k]$ before being returned.

Note: $\text{Node}()$'s default score of -1 is reused both as the "nothing more to add" base-case value *and* as the "not yet computed" memo sentinel. This is a slightly unusual overload - see the callout after the Dry Run section for why it doesn't actually break the answer.

6. Bottom-Up / Tabulation (bottomUp.cpp)

Same recurrence as above, just computed iteratively instead of via recursive calls.

- **Order of computation:** i runs from $n-1$ down to 0, and for each i , k runs from 1 to 4. $\text{dp}[i][k]$ depends only on $\text{dp}[i+1][k]$ (a larger i) and $\text{dp}[j][k-1]$ ($j = \text{nextIdx}[i] \geq i+1$, also a larger i , and a smaller k). Going i from high to low guarantees both dependencies are already filled in before they're needed.
- **Base states handled automatically:** dp is sized $(n+1) \times 5$ and default-constructed, so $\text{dp}[n][*]$ (the $i == n$ base case) and $\text{dp}[*][0]$ (the $k == 0$ base case) are already $\text{Node}()$ {score=-1, idxs={}} from the start - there's no explicit "if base case" check needed, unlike the recursive version.
- **Indexing difference:** instead of $\text{solve}(i+1, k)$ and $\text{solve}(j, k-1)$ function calls, it's simply array reads $\text{dp}[i+1][k]$ and $\text{dp}[j][k-1]$.
- **Syntax note:** the outer "int $k = 4$;" (the fixed number of intervals to choose) is shadowed by the inner loop variable "for (int $k = 1$; $k \leq 4$; $k++$)" inside the main loop. This is valid C++ - the inner k only exists inside that for-loop's scope - and the final "return $\text{dp}[0][k].\text{idxs}$;" correctly refers to the outer k ($= 4$) again. It works, but it's worth renaming one of them for readability.

Net effect: exact same states, exact same values, computed as a filled table instead of a call stack.

7. Dry Run

Intervals (original index: start, end, weight), already sorted by start:

pos i	orig idx	start	end	weight	nextIdx[i]
0	0	1	2	5	2
1	1	1	6	9	4
2	2	3	4	4	3
3	3	5	7	3	4
4	4	8	9	2	5 (=n)

dp[i][k] as (score, idxs) - k needed still, filled i = 4 → 0:

i (start,end,w)	k=1	k=2	k=3	k=4
4 (8,9,2)	1, [4]	1, [4]	1, [4]	1, [4]
3 (5,7,3)	2, [3]	4, [3,4]	4, [3,4]	4, [3,4]
2 (3,4,4)	3, [2]	6, [2,3]	8, [2,3,4]	8, [2,3,4]
1 (1,6,9)	8, [1]	10, [1,4]	10, [1,4]	10, [1,4]
0 (1,2,5)	8, [1]	10, [1,4]	11, [0,2,3]	13, [0,2,3,4]

Reading the last cell: dp[0][4] = (13, [0,2,3,4]) is the final answer's Node. The function returns just the idxs, so the picked intervals are original indices **0, 2, 3, 4**, with a true weight sum of 5+4+3+2 = **14**.

Why the big-weight interval 1 (weight 9) loses: taking interval 1 (1,6) jumps to position 4 (nextIdx[1] = 4), leaving only 1 interval after it - not enough to complete 4 picks. dp correctly reflects this: dp[1][4] tops out at score 10 using only 2 real intervals ([1,4]), which is still less than skipping it and completing a full valid set of 4 for score 13. The DP doesn't need a special "infeasible" flag - running out of intervals just naturally caps how much weight can be added, so it loses on comparison.

About the score being 13, not 14: Node()'s base case score is -1 (also reused as the memo "uncomputed" flag), not 0. Every fully-completed path bottoms out through exactly one k==0 base case, so every completed answer's internal .score field is exactly 1 less than the true weight sum. Since this offset is identical for every completed (i, k) path of the same target k, it never changes *which* option wins a comparison - it only affects the raw .score number. Because maximumWeight() returns only .idxs (never .score), the final answer [0,2,3,4] is still correct.

8. Code

Memoized - memo.cpp

```
#include<bits/stdc++.h>
using namespace std;
class Solution {
public:
    int n;
    struct Node{
        long long score = -1;
        vector<int>idxs;
    };
    vector<int>nextIdx;
    vector<vector<Node>>>dp;
    Node solve(vector<vector<int>>& intervals,int i,int k){
        if(k==0 || i>=n) return Node();
        if(dp[i][k].score!=-1) return dp[i][k];
```

```

        int wt = intervals[i][2];
        int idx = intervals[i][3];
        int j = nextIdx[i];
        Node skip = solve(intervals,i+1,k);
        Node temp = solve(intervals,j,k-1);
        Node take;
        take.score = temp.score + wt;
        take.idxs = temp.idxs;
        take.idxs.push_back(idx);
        sort(begin(take.idxs),end(take.idxs));
        Node result;
        if(skip.score > take.score) result = skip;
        else if(skip.score < take.score) result = take;
        else {
            result = (skip.idxs < take.idxs) ? skip : take;
        }
        return dp[i][k] = result;
    }
}

vector<int> maximumWeight(vector<vector<int>>& intervals) {
    n = intervals.size();
    // store starting index of each interval
    for(int i=0;i<n;i++) intervals[i].push_back(i);
    // sort intervals for start point
    sort(intervals.begin(),intervals.end());
    nextIdx.resize(n);

    for(int i=0;i<n;i++){
        int end = intervals[i][1];
        nextIdx[i] = findNext(intervals,end); //binary search
    }
    int k = 4;
    dp.assign(n+1,vector<Node>(k+1));
    return solve(intervals,0,k).idxs;
}

int findNext(vector<vector<int>>& intervals,int r){
    int low = 0,high=n-1;
    int res = n;
    while(low<=high){
        int mid = low + (high-low)/2;
        if(intervals[mid][0] > r){
            res = mid;
            high = mid - 1;
        }else low = mid+1;
    }
    return res;
}
};

```

Bottom-Up - bottomUp.cpp

```

#include<bits/stdc++.h>
using namespace std;
class Solution {
public:
    int n;
    struct Node{
        long long score = -1;
        vector<int>idxs;
    };
    vector<int>nextIdx;
    vector<vector<Node>>>dp;
    vector<int> maximumWeight(vector<vector<int>>& intervals) {
        n = intervals.size();
        // store starting index of each interval
        for(int i=0;i<n;i++) intervals[i].push_back(i);
        // sort intervals for start point
        sort(intervals.begin(),intervals.end());
        nextIdx.resize(n);

        for(int i=0;i<n;i++){

```

```

        int end = intervals[i][1];
        nextIdx[i] = findNext(intervals,end); //binary search
    }
    int k = 4;
    dp.assign(n+1,vector<Node>(k+1));
    // return solve(intervals,0,k).idxs;
    for(int i=n-1;i>=0;i--){
        int wt = intervals[i][2];
        int idx = intervals[i][3];
        int j = nextIdx[i];
        for(int k=1;k<=4;k++){
            Node skip = dp[i+1][k]; // solve(intervals,i+1,k)
            Node temp = dp[j][k-1]; // solve(intervals,j,k-1)
            Node take;
            take.score = temp.score + wt;
            take.idxs = temp.idxs;
            take.idxs.push_back(idx);
            sort(begin(take.idxs),end(take.idxs));
            Node result;
            if(skip.score > take.score) result = skip;
            else if(skip.score < take.score) result = take;
            else {
                result = (skip.idxs < take.idxs) ? skip : take;
            }
            dp[i][k] = result;
        }
    }
    return dp[0][k].idxs;
}
int findNext(vector<vector<int>>& intervals,int r){
    int low = 0,high=n-1;
    int res = n;
    while(low<=high){
        int mid = low + (high-low)/2;
        if(intervals[mid][0] > r){
            res = mid;
            high = mid - 1;
        }else low = mid+1;
    }
    return res;
}
};

```

9. Complexity

	Time	Space
Sort + build nextIdx (shared)	$O(n \log n)$	$O(n)$
Memoized (memo.cpp)	$O(n * k)$ states, $O(1)$ work each $\Rightarrow O(n \log n)$ overall	$O(n * k)$ dp table + $O(n)$ recursion stack
Bottom-Up (bottomUp.cpp)	$O(n * k)$ states, $O(1)$ work each $\Rightarrow O(n \log n)$ overall	$O(n * k)$ dp table (no recursion stack)

Here $k = 4$ is a fixed constant, so $O(n * k)$ is effectively $O(n)$. The dominant term overall is the $O(n \log n)$ sort + $O(n \log n)$ binary searches for nextIdx. The key practical difference is the memoized version's $O(n)$ recursion stack in the worst case (long chain of skips), while the bottom-up version uses no call stack at all - safer against stack overflow for large n .

10. Quick Revision

- **Core idea:** sort by start, binary-search the first non-overlapping next interval (nextIdx), then it's a plain pick/skip DP.
- **DP state:** $dp[i][k]$ = best {score, idxs} using intervals from position i onward, needing k more picks.
- **Transition:** $dp[i][k]$ = better of $dp[i+1][k]$ (skip) and $dp[nextIdx[i]][k-1] + weight[i]$ (take); tie $\rightarrow$ smaller sorted idx list.
- **Memoized:** recursion + cache keyed by (i, k) ; base case $k==0$ or $i==n$ returns empty.
- **Bottom-Up:** same recurrence, filled iteratively for $i = n-1 \rightarrow 0$, $k = 1 \rightarrow 4$; base cases are just the default-initialized table edges.
- **TC / SC:** $O(n \log n)$ time (sort + binary search dominate; DP itself is $O(n*k)$ with $k=4$ fixed), $O(n*k)$ space (+ $O(n)$ recursion stack for the memo version only).